Proyecto Final. Informe de Avance 2: Detección y Mitigación de Daños Frente a Ataques de Ransomware - Prueba de Concepto

Ivo Gómez Enrico
Departamento Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional
Facultad Regional Santa Fe
Santa Fe, Argentina
igomezenrico@frsf.utn.edu.ar

Francisco Kuchen
Departamento Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional
Facultad Regional Santa Fe
Santa Fe, Argentina
fkuchen@frsf.utn.edu.ar

***Abstract*—Este informe presenta el segundo avance del Proyecto Final orientado a la detección y mitigación de ransomware mediante una Prueba de Concepto que combina la intercepción de E/S vía FUSE con la recuperación basada en snapshots de ZFS. Conforme al cronograma, cubre la implementación avanzada de la herramienta (Guardian FS), el desarrollo del modelo de Machine Learning y la recolección de datos para su entrenamiento. La implementación avanzada incorpora el poder de bloqueo del veredicto del modelo, el estadístico χ^2 por escritura en el score síncrono, un freno global que neutraliza la dilución del scoring en ataques multiproceso, la verificación de identidad de PIDs antes de terminar el proceso atacante, snapshots de emergencia asincrónicos sin shell, y una reducción sistemática de falsos positivos en los archivos señuelo. Para el modelo se adoptó una metodología de evaluación sin fuga de información (validación cruzada agrupada por sesión de recolección, sin sobre-muestreo sintético) junto con un pipeline automatizado de recolección de datos etiquetados en modo shadow. La herramienta supera seis suites de pruebas unitarias con más de 300 aserciones y las validaciones funcionales de detección y de falsos positivos en la VM de laboratorio.**

Palabras Clave—ransomware, detección de intrusiones, FUSE, ZFS, snapshots, entropía de Shannon, Machine Learning, validación cruzada agrupada, falsos positivos, ciberseguridad

I. Introducción

El presente documento constituye el Informe de Avance 2 del Proyecto Final "Detección y Mitigación de Daños Frente a Ataques de Ransomware - Prueba de Concepto", conforme al contenido propuesto en el plan del proyecto: implementación más avanzada de la herramienta y desarrollo del modelo de Machine Learning, junto con la recolección de datos para el modelo.

El proyecto tiene como objetivo evaluar la efectividad de distintas estrategias para la detección de ataques de ransomware y la recuperación de datos, mediante una PoC que intercepta las operaciones de E/S con FUSE y se apoya en snapshots de ZFS para la recuperación [1], [2]. Sobre la primera implementación funcional presentada en el Informe de Avance 1, en esta etapa se completó el endurecimiento de la herramienta (Mes 4 del cronograma), se desarrolló el modelo de Machine Learning con una metodología de evaluación honesta y se construyó la infraestructura de recolección de datos etiquetados para su entrenamiento.

El resto del documento se organiza como sigue: la Sección 2 resume el estado respecto del cronograma; la Sección 3 describe la implementación avanzada; la Sección 4 el desarrollo del modelo; la Sección 5 la recolección de datos; la Sección 6 la validación y resultados preliminares; y la Sección 7 los próximos pasos.

II. Estado de Avance respecto del Cronograma

La siguiente tabla resume el estado de las etapas metodológicas definidas en el cronograma del proyecto al momento de este informe:

TABLA I. Estado de avance respecto del cronograma

Etapa (Cronograma)	***Tareas Principales***	***Estado***
Mes 4: Implementación avanzada	Mejora de la lógica de detección de anomalías e interrupción del proceso maligno.	Completado
Mes 5: Integración de modelo de Machine Learning	Elaboración del modelo de ML e incorporación a la herramienta.	Completado: modelo, metodología de evaluación y recolección de datos
Mes 6: Pruebas	Pruebas exhaustivas contra múltiples familias de malware.	Iniciado: validaciones funcionales preliminares completadas

Síntesis: al cierre de este informe se encuentran completas la implementación avanzada de la herramienta (Mes 4) y la integración del modelo de Machine Learning con su infraestructura de recolección de datos (Mes 5), quedando iniciada la etapa de pruebas (Mes 6) con validaciones funcionales preliminares en la VM de laboratorio.

III. Implementación Avanzada de la Herramienta (Mes 4)

La mejora de la lógica de detección y de la interrupción del proceso maligno se abordó sobre cinco frentes: el enriquecimiento del score síncrono y su integración efectiva con el modelo de ML, la neutralización de técnicas de evasión, la robustez de la mitigación, la reducción de falsos positivos y el realismo del simulador de ataques.

A. Scoring síncrono de cinco señales y poder de bloqueo del veredicto ML

El detector puntúa por PID, en ventanas de 5 segundos, cinco señales normalizadas al rango [0, 1], cada una con peso calibrado:

TABLA II. Señales del score síncrono por PID y ventana

Señal (por PID y ventana de 5 s)	***Peso***	***Rol en la detección***
Entropía media de escrituras	0.35	Huella del cifrado: los datos cifrados presentan entropía cercana al máximo teórico.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Señal (por PID y ventana de 5 s)	*Peso*	*Rol en la detección*
Estadístico χ^2 medio (uniformidad)	0.20	Discrimina cifrado de compresión: un archivo comprimido tiene alta entropía pero distribución no uniforme.
Tasa de escrituras	0.20	El cifrado masivo exige tasas sostenidas que el uso legítimo no alcanza.
Tasa de renombrados	0.15	Cambio de extensión (.docx → .locked), comportamiento clásico de las familias de referencia.
Tasa de unlinks (caza de backups)	0.10	Borrado de copias de seguridad: análogo Linux del vssadmin delete shadows de Windows.

Un score mayor o igual a 0.65 produce el bloqueo de la operación y mayor o igual a 0.45 el estado de sospechoso. Se corrigieron dos defectos estructurales del score: (1) el estadístico χ^2 se calculaba sobre un histograma acumulado que ninguna ruta de código poblaba, por lo que el 20 % del peso estaba siempre en cero y un ataque de solo-entropía (sin renombrados ni canaries) jamás alcanzaba el umbral de bloqueo; ahora el handler FUSE pasa el χ^2 real de cada escritura y el detector acumula su media por ventana; (2) el veredicto de ataque emitido por el modelo de ML (segunda opinión asíncrona) no tenía poder de cumplimiento: la marca de ataque confirmado se establecía pero ninguna ruta de decisión la consultaba. Ahora las escrituras y renombrados posteriores de un PID con ataque confirmado reciben bloqueo directo.

Adicionalmente, la tabla de estado por PID se acotó a 256 entradas con liberación de las inactivas (más de 30 segundos sin eventos): sin este límite, estado peligroso (p. ej., la señal de canary armada) sobrevivía al reuso del PID por un proceso inocente.

B. Detección anti-evasión: freno global frente a ataques multiproceso

El scoring por PID es vulnerable a la dilución: un ransomware que bifurca N procesos trabajadores mantiene a cada PID por debajo de todos los umbrales. Para neutralizarlo se incorporó un freno global que agrega, por ventana, las escrituras con firma de cifrado (entropía por encima del umbral y χ^2 menor a 300, esto es, distribución uniforme) de todos los PIDs. Cuando el agregado alcanza 100 escrituras (o 60 renombrados con cambio de extensión), el freno se arma y los escritores con firma de cifrado reciben bloqueo aunque su score individual no llegue al umbral. Un proceso benigno de baja entropía nunca es bloqueado por el freno, y la exigencia de uniformidad excluye a los archivos simplemente comprimidos (p. ej., un directorio de fotografías o de copias comprimidas), que presentan entropía alta pero χ^2 alto [3].

C. Mitigación robusta: verificación de identidad y snapshots asincrónicos

La terminación del proceso atacante se endureció con verificación de identidad: al crearse la entrada de un PID, el detector registra su tiempo de inicio (campo 22 de /proc/<pid>/stat), y antes de emitir el SIGKILL se compara con el valor actual. Si el PID fue reutilizado por otro proceso, la terminación se omite y el estado envenenado se descarta. Esto cierra la ventana que abre el veredicto asíncrono del modelo, que llega hasta 5 segundos después de los eventos. El resultado de cada terminación queda registrado en el log estructurado de eventos (process_killed / kill_skipped).

La capa de snapshots pasó de invocar zfs(8) de forma síncrona mediante system() dentro del handler de escritura (del orden de 100 a 300 ms por escritura bloqueada, y sin deduplicación ante un atacante multihilo) a un hilo trabajador señalizado con un período de enfriamiento de 10 segundos. Todas las invocaciones de la CLI de ZFS se realizan ahora vía posix_spawnp con vector de argumentos directo, eliminando el shell y la interpolación de cadenas. Adicionalmente, el snapshot de emergencia se dispara también en el primer veredicto de sospecha (pre-daño), preservando un estado casi limpio, y la retención se acotó a los últimos 20 snapshots automáticos y 10 de emergencia [1].

D. Reducción de falsos positivos: canaries por intención de acceso

En la versión anterior, cualquier apertura de un archivo señuelo armaba la terminación del proceso, de modo que un cp -r, tar o rsync que recorriera canaries dentro del punto de montaje moría en su primera escritura posterior. La señal se discrimina ahora por la intención de acceso: la apertura para escritura (O_WRONLY / O_RDWR, comportamiento de ransomware) arma la señal; la apertura de solo lectura (copias de seguridad, generadores de miniaturas, antivirus) aporta únicamente una señal leve. La regla armada, además, bloquea solo escrituras con firma de cifrado: un usuario que edita el señuelo (grabados de baja entropía, archivos temporales del editor) no es terminado. El renombrado de un canary se discrimina por cambio de extensión (.docx → .locked arma la señal; una reorganización de archivos, no).

Se corrigió asimismo el despliegue de señuelos: se solicitaban 20 pero se desplegaban 7, concentrados en la raíz del dataset; ahora se despliega la cantidad efectiva distribuida entre la raíz y subdirectorios, cubriendo también los recorridos recursivos del cifrado.

E. Caza de backups y modo shadow

La tasa de unlinks se incorporó al score (Tabla II) como señal de caza de backups, el análogo Linux del borrado de copias de sombra de Windows: sola no bloquea (un make clean aporta como máximo 0.10, por debajo del umbral 0.65), pero cifrar y luego borrar los originales sí lo hace [4].

Se agregó el modo shadow (GUARDIAN_SHADOW_MODE): el daemon registra los veredictos de bloqueo sin ejecutar la mitigación (sin EPERM, sin SIGKILL, sin snapshot). Esto habilita dos usos: la recolección de datos de entrenamiento sin interrumpir los ataques simulados (Sección 5) y la medición de falsos positivos sobre cargas reales sin riesgo. Los eventos se registran con el modo explícito, y un evento inicial del daemon documenta la configuración vigente (umbrales, ventana y modo), dando trazabilidad al dataset producido.

F. Simulador con perfiles de familias reales

El simulador de la fase de calibración se extendió con patrones de E/S documentados de las familias de referencia (LockBit, Cl0p, RansomEXX):

TABLA III. FLAGS DEL SIMULADOR Y PERFILES DE FAMILIAS REALES

Flag del simulador	*Qué simula*	*Por qué importa*
--workers N	N procesos paralelos, cada uno con su PID, atacando subconjuntos disjuntos de archivos.	Ejercita la dilución del scoring por PID (evasión multiproceso real).

Flag del simulador	*Qué simula*	*Por qué importa*
--partial-encrypt PCT	Cifrado intermitente: solo los primeros PCT % de cada archivo, escrito en bloques de 256 KB.	Reduce los bytes cifrados por archivo y pone a prueba los umbrales de tasa (patrón de LockBit).
--realistic	Tamaños log-uniformes de 16 KB a 8 MB, jitter en pausas y ~5 % de archivos salteados.	Distribuciones de un filesystem real en lugar de cargas planas.
--mmap	Escrituras vía mmap en lugar de write().	Prueba de evasión: documenta si el detector observa el volcado de páginas sucias.
--seed N	Semilla de aleatoriedad de toda la corrida.	Corridas reproducibles y comparables entre configuraciones.

IV. Desarrollo del Modelo de Machine Learning (Mes 5)

A. Pipeline de inferencia y features

El hilo analyzer calcula 14 features por PID y ventana de 5 segundos y las envía por Unix socket al servidor de inferencia (ml_server.py): estadísticas de entropía (media, máximo, desvío y autocorrelación lag-1), tasas de escrituras, bytes escritos, renombrados y unlinks, ratio lectura/escritura, estadístico χ^2, tasa de cambio de extensión, acceso a canaries, directorios únicos y variedad de tipos de archivo. El servidor responde con un veredicto (normal / suspicious / attack); en ausencia de modelos entrenados degrada a reglas estadísticas, por lo que el camino feliz de detección (el score síncrono en C) no depende del ML.

B. Ensemble de modelos

El servidor combina Random Forest (peso 0.35) y XGBoost (0.30) como clasificadores supervisados, Isolation Forest (0.20) entrenado únicamente con la clase benigna como detector de anomalías, y opcionalmente una LSTM (0.15) sobre la secuencia de ventanas del PID. Se corrigieron dos defectos: la LSTM solo participa si existe un checkpoint entrenado (antes se construía con pesos aleatorios si PyTorch estaba instalado, inyectando ruido en el ensemble), y los pesos se renormalizan con los modelos realmente disponibles, pues un peso muerto sesga el umbral de decisión. El veredicto de ataque del ensemble alimenta la marca de ataque confirmado del detector (Sección 3.1).

C. Metodología de evaluación sin fuga de información

El script de entrenamiento fue reescrito con una metodología que garantiza métricas honestas. La Tabla IV resume los problemas de la evaluación previa y las correcciones adoptadas:

TABLA IV. Metodología de evaluación: problemas y correcciones

Problema en la evaluación previa	*Corrección adoptada*
SMOTE aplicado a todo el dataset antes de la validación: sintetizaba muestras dentro de los folds de test e interpolaba features binarias o enteras (canary_accessed = 0.37 carece de sentido físico).	Sobre-muestreo eliminado; el desbalance se maneja con class_weight (Random Forest) y scale_pos_weight (XGBoost) [5].
StandardScaler ajustado con todo el dataset antes del CV: estadísticos del test filtrados al entrenamiento.	El scaler vive dentro del Pipeline: se ajusta solo con el fold de entrenamiento.
Validación cruzada aleatoria: ventanas contiguas del mismo ataque repartidas entre entrenamiento y test.	StratifiedGroupKFold agrupada por sesión de recolección; holdout también agrupado (GroupShuffleSplit).
Hiperparámetros duplicados entre el entrenamiento y la inferencia (dos rutas de código divergentes).	El entrenamiento usa la misma clase que sirve la inferencia: lo evaluado es lo desplegado [6].

V. Recolección de Datos para el Modelo

A. Registro de features con sesiones de recolección

El proxy de features (ml_proxy.py) se interpone entre el analyzer y el servidor de inferencia y registra cada vector en el CSV de entrenamiento. La columna de sesión (asignada por ronda mediante el flag --tag) es la que habilita la validación agrupada de la Sección 4.3: las ventanas de una misma corrida no se reparten entre entrenamiento y test.

B. Campaña automatizada en modo shadow

El script collect_training_data.sh orquesta la campaña en dos fases: (1) ataques: rondas del simulador variando modo (full / fast / stealth), cantidad de archivos, pausas, workers (1 a 4), porcentaje de cifrado parcial (50 a 100 %) y perfil realista; (2) benignos: copias de código y documentación, empaquetado tar.gz, appends a logs, CSVs y copias de binarios. El daemon FUSE opera en modo shadow durante toda la campaña, por lo que los ataques generan features de todas sus ventanas en lugar de morir en la primera, y el proxy se reinicia por ronda con etiquetas de sesión atk_rN / ben_rN. Toda la operación se confina a directorios de /tmp, sin tocar datos reales.

Se estima del orden de 2 a 3 horas para una campaña de 100 rondas por clase, desatendida, con un requerimiento aproximado de 2 GB libres en /tmp (los archivos renombrados se acumulan hasta la limpieza final).

C. Formato del dataset y estado de la campaña

TABLA V. Formato del dataset de entrenamiento (CSV)

Columna(s) del CSV	*Contenido*
1-14	Features de comportamiento por PID y ventana de 5 s (Sección 4.1).
15 (label)	Clase: 0 = benigno, 1 = ataque (por fase de la campaña).
16 (session)	Ronda de recolección (atk_rN / ben_rN): habilita la evaluación agrupada.
17-18 (timestamp, pid)	Trazabilidad de la ventana registrada.

Estado: la infraestructura de recolección fue validada en la VM de laboratorio mediante las corridas integrales del pipeline (smoke test con generación de filas y etiquetado por sesión verificados); la campaña completa (100 o más rondas por clase) se ejecutará como parte de la etapa de pruebas para producir el dataset de entrenamiento definitivo.

VI. Validación y Resultados Preliminares

A. Pruebas unitarias

La suite de pruebas unitarias se extendió con las regresiones de cada corrección (freno global, caza de backups, canary por intención, verificación de identidad, descarte de estado envenenado):

TABLA VI. Resultados de las suites de pruebas unitarias

Suite (módulo)	*Aserciones*	*Resultado*
entropy (Shannon, χ^2, ventana deslizante, autocorrelación)	12	Exitoso
detector (score, freno global, unlinks, canaries, identidad)	23	Exitoso
canary (despliegue distribuido y reconocimiento)	14	Exitoso
ring_buffer (cola productor-consumidor)	46	Exitoso
mitigation (identidad anti reuso de PID)	5	Exitoso
analyzer (features por PID y ventana)	218	Exitoso

Más de 300 aserciones en seis suites, todas exitosas. Cabe destacar la prueba de mitigación más restrictiva: el propio proceso de test invoca la terminación contra su PID con un tiempo de inicio incorrecto y debe sobrevivir (es la verificación de identidad funcionando como último recurso de seguridad).

B. Validación funcional en la VM

Las validaciones funcionales preliminares en la VM de laboratorio confirman: (1) detección operativa: el smoke test del pipeline completo (test_ml_pipeline.sh) termina con el simulador muerto por la mitigación (SIGKILL tras el bloqueo de la escritura); (2) anti-falsos-positivos: un cp -r y un grep recursivo que recorren canaries dentro del punto de montaje completan sin ser terminados (antes de las correcciones de la Sección 3.4, morían en su primera escritura posterior). Quedan para la campaña de pruebas del Mes 6: la validación del freno global con ataques multiproceso, la verificación empírica del vector mmap, el rollback de extremo a extremo y las métricas de la PoC.

VII. Próximos Pasos

1. Campaña de recolección completa (100 o más rondas por clase) y entrenamiento del modelo con la metodología de la Sección 4.3. Se espera un ROC-AUC más conservador que en evaluaciones previas, como consecuencia de eliminar la fuga de información.
2. Calibración de umbrales y pesos del detector síncrono a partir del dataset real (hoy calibrados manualmente).
3. Fase de destrucción (Mes 6): ejecución de las muestras reales de las familias de referencia en la VM aislada, y verificación empírica del vector de evasión mmap.
4. Medición de las métricas de la PoC: pérdida de datos antes del bloqueo (< 1 MB objetivo), RTO del rollback (< 30 s), overhead de snapshots (< 5 %) y latencia de E/S inducida por FUSE.

Se mantiene vigente el Riesgo 1 (degradación de rendimiento por la intercepción FUSE), mitigado por el prototipado evolutivo y el despliegue confinado en VM; el Riesgo 3 (ineficacia frente a muestras no perfiladas) se ataca en el Mes 6 con la campaña sobre múltiples familias, y el vector mmap identificado se gestiona con la contingencia ya definida en el plan (degradar FUSE a un modo asíncrono de auditoría).

Referencias

[1] V. Dakic, M. Kovac, and I. Videc, "High-performance computing storage performance and design patterns—Btrfs and ZFS performance for different use cases," Computers, vol. 13, no. 6, p. 139, 2024, doi: 10.3390/computers13060139.

[2] S. Lee et al., "Rcryptect: Real-time detection of cryptographic function in the user-space filesystem," Computers & Security, vol. 112, p. 102512, 2021, doi: 10.1016/j.cose.2021.102512.

[3] P. O'Kane, S. Sezer, and D. Carlin, "Evolution of ransomware," IET Networks, vol. 7, no. 5, pp. 321–327, 2018, doi: 10.1049/iet-net.2017.0207.

[4] C. Beaman et al., "Ransomware: Recent advances, analysis, challenges and future research directions," Computers & Security, vol. 111, p. 102490, 2021, doi: 10.1016/j.cose.2021.102490.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.